

Estructuras de programación

Objetivos

- Comprender la diferencia entre análisis léxico, sintaxis y semántica.
- Identificar los componentes léxicos de Java.
- Construir y evaluar expresiones matemáticas y lógicas.
- Utilizar métodos de la biblioteca estándar de Java.
- Reconocer los distintos tipos de sentencias.
- Entender el concepto de flujo de control.
- Desarrollar algoritmos iterativos.
- Realizar trazas de ejecución.

Estructuras de programación

Objetivos

- Comprender la anatomía básica de un programa y sus elementos constituyentes.
 - Entender la diferencia conceptual entre sintaxis y semántica.
 - Introducir el concepto de token, palabras reservadas y expresiones.
 - Comprender la necesidad de alterar el flujo de ejecución secuencial mediante estructuras de control y modularidad.

Introducción

Anteriormente hemos visto que, para poder ejecutar un programa, es estrictamente necesario traducirlo a código máquina. En ese proceso inicial, aprendimos que Java es un lenguaje que se fundamenta en la portabilidad y la seguridad, utilizando el *bytecode* y la Máquina Virtual de Java (JVM) como intermediarios indispensables para que nuestras instrucciones lleguen, finalmente, a ser procesadas por el hardware.

Sin embargo, programar no consiste simplemente en memorizar y agrupar un conjunto de comandos aislados. Un lenguaje de programación es, en esencia, un **sistema formal** compuesto por un conjunto de reglas, símbolos y palabras especiales que se combinan para construir un algoritmo capaz de resolver un problema real. Por ello, para avanzar de forma sólida en nuestro aprendizaje, debemos empezar a analizar el texto de nuestros programas buscando e identificando los elementos mínimos que poseen un significado especial para el compilador.

Uno de estos elementos fundamentales son las **instrucciones** o **sentencias**. De la misma forma que en la comunicación verbal natural utilizamos frases u oraciones estructuradas mediante sintagmas, las instrucciones de un lenguaje de programación contienen piezas estructurales menores que denominamos **expresiones**. Siguiendo con

esta analogía lingüística, al igual que el predicado de una oración necesita casi siempre de un verbo para dotar a la frase de acción y sentido, las expresiones en programación se construyen habitualmente articulando variables, literales y **operadores**.

En este tema, daremos un paso más allá de la simple declaración de datos y la ejecución estrictamente lineal. Abordaremos de forma estructurada los siguientes conceptos:

- **Análisis léxico:** Cómo el compilador lee nuestro código aislando **palabras reservadas** y símbolos (tokens).
- **Construcción de expresiones:** Cómo se articulan y evalúan operaciones complejas de cálculo y lógica.
- **Modularidad:** Cómo podemos organizar bloques lógicos reutilizables mediante el uso de **funciones** o métodos.
- **Control de flujo:** Cómo romper la secuencialidad, sentando las bases para que nuestros programas puedan tomar decisiones lógicas (selección) y repetir procesos de manera eficiente (iteración).

Dominar estos conceptos fundacionales es el requisito indispensable para pasar de escribir simples líneas de código a diseñar soluciones lógicas y estructuradas.

Estructura de programación: Conjunto de reglas semánticas y sintácticas que permiten organizar y articular las instrucciones de un programa para controlar su flujo de ejecución, evaluar expresiones complejas o definir bloques de código reutilizables.

Nota de diseño de imagen: > **Figura sugerida:** Un diagrama de bloques que muestre el esqueleto de un programa. En la parte superior, un bloque principal ("Código Fuente"). De él cuelgan ramificaciones hacia "Palabras Reservadas", "Expresiones" (con subramas hacia "Operadores" y "Variables"), y "Control de Flujo" (con flechas que ilustren bifurcaciones lógicas y bucles). El objetivo es visualizar la anatomía estructural que se desarrollará en las siguientes secciones.
